ILLUMINARY LABS

KNOAD Wallet

KNOAD keeps you safe from intrusion by ending the standing access every other wallet leaves in place.

Standing access is the intrusion
Every other wallet authorizes one key for the life of the account. Whoever holds that key is in, and they stay in.

KNOAD ends that access
The key that can spend does not keep its seat. A leak is not a permanent place inside the wallet.

You do not have to start over
Cutting an intruder off does not mean a new wallet, or moving what you already hold.

We do not hold a way in
The way back is yours. We never see it, and we cannot open the wallet for anyone.

Technical white paper · 16 October 2026 · Pre-audit. This is an explanation of the implementation, not a security certification, and not a clearance to put assets of value on these contracts.

How to read this

The synopsis is the full easy read. It is enough on its own. The chapters after it tell the same story and put the comparisons, the maps, and the measurements next to each part. The appendices are lookup cards. You do not need one to understand the paper.

Jump if you already know the question.

Synopsis | Using it | Recovery | Fees | If KNOAD is gone | What this is not | Appendices

> Spec 1.4 is the normative contract text. This paper is the explanation. Spec 1.3, spec 1.2, and spec 1.1 remain the previous texts. The PDF named v1.0 is the earlier design draft.

Contents

Synopsis 3
1. What you are holding 5
2. Why the split exists 6
3. Using it 8
4. What an included transaction does 9
5. Recovery and the optional waits 11
6. What rotation does not erase 13
7. Optional commerce 14
8. If KNOAD is gone 15
9. What this is not 16
10. Where the work stands 18
Appendices 20
- Appendix A. Glossary 20
- Appendix B. Chains and published addresses 20
- Appendix C. Derivation 21
- Appendix D. Contract behavior 21
- Appendix E. Commerce checks 22
- Appendix F. Residual risk 23
- Appendix G. What changed from draft 1.2 23

Synopsis

KNOAD keeps you safe from intrusion by ending the standing access every other wallet leaves in place.

That ending happens on the chain that includes the next operation. It does not, by itself, retire the key on the other chains. It does not erase a token approval. It does not make the 24 words optional, and it does not run on a timer.

An ordinary wallet authorizes one key for as long as the wallet exists. The key and the wallet are the same thing. If that key leaks, the intruder has the same standing you do, and they keep it. Cutting them off means a new wallet. Every token, NFT, name, and approval attached to the old one has to be moved by hand.

KNOAD separates the wallet from the key that can spend today. The wallet stays. The spending key does not keep its seat. The phone prepares the next key and saves it before it signs. The signature covers both the action you asked for and that next key. When the network includes the operation, the old key is finished. That holds even when the payment itself fails. The app treats “signed,” “included,” and “the payment went through” as three different facts.

The 24 words are not the daily key. They are the way back. KNOAD never receives them, and getting the wallet back does not call KNOAD. On a new phone, the words find the wallet and install a new spending key. Anyone can pay the network fee for that transaction, so someone who has already taken the wallet’s ETH cannot price you out of coming back. The spending key cannot change the way back.

The address is the same on Ethereum, Base, Sepolia, and Base Sepolia. It is known before the account is deployed. Funds sent there, including funds bridged there, arrive at a wallet these words control, including on a chain where the account has not been deployed yet. The address is fixed by the original recovery authority, the account index, and the empty creation policy. The first spending key is not part of it. The first included send on a chain installs that chain’s own first key, and only if the original recovery key signed that key for that chain. A later rotation or recovery on one chain does not rewrite the address, and it does not move the key on the others.

Each chain has its own first spending key, from the first send. A rotation writes only the chain that included it. There is no clock, and a signature on one chain does not publish the key for another. A chain that has not been set up can already receive. It gets its key when the first send on that chain is included.

The 24 words are how you take a phone out of the wallet. You enter them once, on a phone you still have. That sitting can look at the address and the balances without a transaction, and it can prepare every chain the phone can read, including a chain several steps behind. The account still accepts exactly one recovery step at a time. The phone signs each step while the words are on screen, and each step is paid in order. The words are cleared when you finish, go back, or leave that screen. They are not saved. Anyone can pay the network fee for a recovery, so someone

who has already taken the wallet's ETH cannot price you out of coming back. Until a chain includes the switch, the previous phone can still spend on it, and can still get a transaction in first. What lands first stays moved. A chain that was never set up can be opened by whoever lands first. Approvals already granted in token and NFT contracts stay granted. A signature names its chain, so one chain cannot switch another.

The phone is the only signer. A site paired over WalletConnect sees the address and the chain. The browser extension only relays that pairing. It does not hold the words, a spending key, or a trading screen.

Create. The phone shows 24 words once and asks you to write them down. They are not saved on the phone. A separate app password locks the vault on that device. The password is not the way back. From the words, the phone computes the address. Nothing has to be deployed before that address can receive funds. Each chain gets its own key on the first send.

Unlock. The password opens this phone. It does not recover a wallet, and it is not the phrase.

Receive. The wallet screen lists ETH, WETH, and USDC on the chain you have selected. Other tokens can still sit at the address. Those three are the ones the app lists, and the ones a swap or a bridge can select.

Send. You name a recipient. The phone saves the next spending key, signs, and submits when a bundler is configured. Send does not add a KNOAD fee. The note after signing says the old key is retired on that chain only once the network includes the operation, and that a failed payment can still retire it there. A waiting send can be sped up or cancelled with the same next key. The old key stays until one of those is included. The screen distinguishes a payment that failed after the key changed from a send that never landed. The other chains are unchanged until each includes its own operation.

Pair a site. From Security, you pair a WalletConnect session. The site sees the address and can ask the phone to sign or to send. The signing sheet is on the phone, outside the page. Disconnecting ends that pairing. It does not take back a token approval the site already received.

Look at a price. Swap and Bridge ask for a price in ETH, WETH, or USDC. The sheet shows four numbers side by side: what you receive, the network fee, the provider's fee, and KNOAD's fee. It also shows the route, the slippage, each approval, and a simulated balance change when the network can run the whole batch. Confirming does not send. Buy / Sell and NFTs are on the phone and do not transact.

Come back on a new phone. You enter the 24 words once, and a phone password. Looking finds the address and can show balances with no transaction. Making this phone the one that can spend prepares every chain it can read, including a chain several steps behind. The account still takes one step at a time. The phone signs each step in that sitting, and anyone can pay a chain that is already deployed. On a chain that has never been set up and that holds native funds, this phone submits the first use itself, paid from that balance, and installs a new key. A chain that holds only tokens still needs someone to pay. Whoever lands first chooses the key. Restoring the

words does not cut off the old phone. Recovery does. Secure all networks is optional. It lists every chain, and someone can pay the network fee to close an empty chain before anything arrives. The words are cleared when you finish, go back, or leave that screen. They are not written into the vault. A chain that could not be checked is left as it is. This phone does not spend on a chain that has moved on until that chain has included its own switch. Until then, the other phone can still spend there. This version derives with an empty BIP-39 passphrase, because the phone never asks for one. A passphrase would have to be chosen before the address exists.

Creating a wallet, restoring one, sending, receiving, retiring a spending key, recovering, managing permissions, and connecting a site do not pay KNOAD. Network gas is still yours. Nobody else pays that gas for you. An optional swap or bridge can include a KNOAD fee of 0.25 percent. It cannot be set above 0.30 percent. If no fee recipient is configured, that line is zero. The official app shows those quotes and does not broadcast them. Submission stays off until the account has been independently reviewed.

This does not make the words optional. Someone who has them has the way back, and the chain cannot tell you from a thief holding the same words. A compromised phone can still sign a theft while its spending key is current. Token approvals and NFT operator grants live in other contracts, so retiring a spending key does not erase them. The keys are still ordinary elliptic-curve keys. This is not post-quantum security, and this paper is not an audit.

If KNOAD disappeared, the address could still be recomputed from the words and the published factory. Sending and recovery do not call a KNOAD server. Another bundler can submit a signed operation. Only swap and bridge prices would disappear. There is no company key that can freeze the wallet, replace the signer, or upgrade it.

The system described here is built. Real assets wait on an independent review of the account, the factory, the recovery check, and the phone that holds the keys. Until that review, these contracts are not cleared to hold assets of value.

The chapters after this one are the same story with the comparisons, the maps, and the measurements. You do not need them to understand what you are holding.

1. What you are holding

Four pieces exist. Each one is allowed to know a different thing. The quote service can be down, and send, recovery, and pairing still work.

Who is allowed to know what

Phone
The wallet. It holds the spending keys, shows the signing sheet, and submits. It does not keep the 24 words after you write them down.

Extension
A relay for pairing a site. It learns the address. It cannot sign, and it has no trading screen.

Quote service
Asks for swap and bridge prices. The provider keys stay on that server. The phone does not trust the answer.

Account
Holds the assets and decides which key may spend. It does not store a phrase, and it has no company admin key.

Factory
Deploys the account at an address that can be computed before deployment. The same inputs produce the same address on every chain in the registry.

Chains
Ethereum, Base, Sepolia, and Base Sepolia. One address across all four.

The account and the factory. The account is a smart contract. It holds assets and decides which key may spend. The factory deploys that account at an address that can be computed in advance from the original recovery key, the account index, the empty creation policy, and the published factory. The first spending key is not part of that address. The same inputs produce the same address on every chain in the registry. The contracts do not store a seed, and they do not have an Illuminary Labs admin key.

The phone app. This is the wallet. It creates the phrase, keeps the spending keys, shows the signing sheet, and submits transactions. The phrase is shown once, for the person to write down. After that, the daily vault stores what the phone needs to spend. It does not store the phrase. On a device that can do it, later spending keys stay in the secure hardware and are not exportable. The app password unlocks that vault. It is not an input to recovery.

The browser extension. This relays WalletConnect sessions to the phone. A paired site learns the account address. The extension cannot sign, and it cannot display a swap or a bridge.

The quote service. This asks 0x and LI.FI for swap and bridge prices, using API keys that stay on the server. The phone does not trust the answer. It checks the calls itself before it will sign. If this service is down, only Swap and Bridge are affected. Send, recovery, and pairing keep working.

Buy / Sell and NFTs are present in the phone as inactive screens. They cannot request a quote and they cannot ask for a signature.

2. Why the split exists

An externally owned Ethereum account has one private key, and the address is that key. The model is simple and portable. It has no way to retire a spending key without creating a different

address and moving what the old address holds. That becomes painful for someone who has collected NFTs, names, approvals, and signed orders at one address.

A smart account makes the address a contract, and makes authorization a piece of state the contract can replace. Many of those accounts still leave standing access in place: a long-lived signer, a module that can act on its own, or a guardian set that outlives any one payment. The cost of a contract account is that the contract's code now sits under everything the person owns. KNOAD keeps that code small. There is no module installer and no general "execute anything" entry that a hot key can aim at the account itself.

The comparison below is about who stays authorized. It is not a market ranking, and it does not assign numbers to other products.

Model	Who stays authorized	What a leak means
Ordinary wallet	One key, for the life of the account	The intruder stays. Cutting them off means a new address and a move.
Smart wallet with a long-lived signer or guardians	A signer, a module, or a guardian set that outlives one payment	Standing access moves. It is not removed.
KNOAD	One spending key on a chain, replaced when that chain includes an operation	The leaked key does not keep its seat on that chain. The wallet stays. Other chains keep their own current key until each includes an operation.

How long a leaked key remains useful

Ordinary wallet
From the moment it leaks until the owner abandons the address and moves what it holds.

Long-lived smart wallet
For as long as that signer, module, or guardian set remains authorized.

KNOAD, on the chain that includes the next operation
Until that inclusion. The old key is finished then, including when the payment itself fails. There is no timer. A key that is still current can still sign.

KNOAD, on a chain that has not included one
That chain keeps the key it already has, including its own first key before the account is deployed. The address can receive before it is set up. The chain stays on that key until its own send or its own recovery is included.

Recovery authority
One entry of the words prepares every chain the phone can read, including every step of a chain that is several behind. Each step still needs its own included transaction, in order.

Three roles follow from that.

Role	What it is	What it cannot do
Address	The account. Stable across key changes and across the four chains.	It is not a key.

Role	What it is	What it cannot do
Hot signer	The spending key in use right now. Exactly one is current.	It cannot change the recovery key, the recovery verifier, the spend policy, or the implementation.
Recovery authority	An address derived from the 24 words.	Ordinary sends do not touch it.

The hot signer spends. The recovery authority replaces a hot signer that is lost or no longer trusted. Those are different secrets on purpose. A leak of today's spending key is not a leak of the phrase, unless the same device was also holding the phrase.

3. Using it

The order below is the order a person meets the product. At each step, three outcomes stay distinct: the phone signed, the network included the transaction, and the underlying action succeeded.

What you see, and what the chain records

Signed

The phone produced a signature. The next spending key is already saved on the device. Nothing on the chain has changed yet.

Included

The network accepted the operation. The old spending key is retired. This can happen even when the payment fails.

The payment went through

The calls did what you asked. This is a separate fact from inclusion. A retired key is not proof that the funds moved.

Create. The phone generates 24 words and asks the person to write them down. They are not saved on the phone. A short check confirms one of the words. A separate app password encrypts the device vault. From those words the phone derives the recovery key and a one-time first spending key for each chain, signs that chain's activation, and stores the key and the signature in the vault. The words are then discarded. Nothing is deployed yet. The address can already receive funds. The activation has no deadline. It dies when that chain is deployed. Restoring the words rebuilds the same activation.

Unlock. The password opens the vault on this phone. It does not recover an account, and it is not the phrase.

Receive. The address is the same on Ethereum, Base, Sepolia, and Base Sepolia. The Wallet screen shows ETH, WETH, and USDC on the selected chain. Other tokens can still be held; those three are the ones the app lists and the ones a swap or bridge can select.

Send. The person enters a recipient. The phone saves the next spending key, signs one operation, and submits it when a bundler is configured. The note after signing says the key

rotation counts only after the network includes the operation on that chain. A failed payment can still retire the key there. The app says so, rather than rolling the two into one success message. A waiting send can be sped up or cancelled with the same saved next key. The old key stays until one of those is included. Send does not add a KNOAD fee.

Pair a site. From Security, the person pairs a WalletConnect session. The site sees the account address and can ask the phone to sign or to send calls. The signing sheet is on the phone, outside the page. Disconnecting the session ends that pairing. It does not revoke a token approval the site already obtained.

Look at a quote. Swap and Bridge ask the quote service for a price in ETH, WETH, or USDC. The confirm sheet lists the amount to be received and the minimum, the network fee, the provider fee, the KNOAD fee, the route, the slippage, and each approval. It also shows a simulated balance change when the network can simulate the whole batch. Confirming does not submit. The sheet says submission opens after the account review. Buy / Sell and NFTs explain that they are not active.

Recover on a new phone. The person enters the 24 words once and a phone password. Looking up the wallet finds the address and can show balances with no transaction. Making this phone the one that can spend prepares every chain it can read, including a chain several steps behind. The account still accepts one step at a time, so the phone signs each step in that sitting and anyone pays them in order. It does not jump an epoch in one call. On a chain that has never been set up and that holds native funds, this phone submits the first use itself, paid from that balance. A chain that holds only tokens still needs someone to pay. Whoever lands first chooses the key. Restoring the words does not cut off the old phone. Recovery does. Secure all networks is optional. It lists every chain, and someone can pay the network fee to close an empty chain before anything arrives. The words are cleared when the person finishes, goes back, or leaves that screen, and they are not saved. A chain that could not be checked is left as it is. The new spending key is kept aside until that chain’s own transaction is included. Until then, the previous phone can still spend there.

4. What an included transaction does

The hot path is one sequence. The phone saves the next key before it signs. The signature covers the action and the next key together. Inclusion is what retires the old key, and that retirement happens before the calls run.

What one included operation does

> **Save the next key**
> The phone writes it to durable storage, with the generation it belongs to, before anything is signed.
>
> **Sign both**
> The calls and the next key are one payload. There is no unsigned next key beside the signature.
>
> **The chain includes it**
> A bundler’s “received” is not inclusion. The account has one transaction lane, and the phone reads the next sequence from the account.
>
> **The old key is retired**
> Validation installs the next key and advances the generation before the calls run.
>
> **The calls run**
> If they revert, the rotation still stands. If validation itself fails, nothing rotates.

Before the phone produces a signature, it writes the next spending key to durable storage, together with the generation that key belongs to. A crash between signing and saving would otherwise leave the account on a key the phone no longer has. Recovery still works in that case. The phone treats it as a recovery, not as a retry with the old key.

The signed operation names the calls and the next signer in the same payload. The signature covers both. There is no unsigned “next key” attached beside the signature.

Inclusion is decided by the chain, not by a bundler’s acceptance message. The account’s generation is the sequence number of its one transaction lane. The phone reads that generation from the account. It does not keep two operations in flight that name different next keys at the same nonce.

When validation of an included operation succeeds, the account installs the next signer and advances the generation before the calls run. Each operation carries its own validation binding, so two operations from the same account in one bundle do not cancel each other, and a recovery that lands before the calls run stops those calls. If the calls then revert, the network still keeps the rotation, because the rotation already happened in validation. If validation itself fails — wrong key, wrong nonce, zero next signer — nothing rotates. Validation does not read the clock.

A batch holds at most eight calls. Calls to the zero address are rejected. A call back into the account itself is rejected unless it is the specific function that grants or revokes an order digest. A key refresh is an operation with no calls: the execution payload is empty, and the key still rotates. An encoded list of zero calls is rejected. A waiting send can be replaced at the same nonce with a higher fee, or cancelled with that empty payload, and both reuse the next key already saved. The old key stays current until one of those operations is included. The screen distinguishes a payment that failed after the key changed from a send that never landed.

Two optional limits exist, both off unless the owner turns them on. They count native ETH attached to the calls. They do not count token amounts.

- A period limit caps how much ETH the hot key can attach in a time window.
- A delay threshold queues a batch whose ETH value is above the threshold. The first inclusion records the call hash and rotates the key without moving the funds. A later operation with the same calls, after the delay, executes them.

A swap or bridge quote that would cross either limit is refused before signing. Letting it through would burn the nonce: the key would rotate, and the trade would revert, or the calls would sit queued and run later on a route whose price had already expired. The phone tells the person to use Send for a large native move.

Gas limits and fees on a real submission come from the bundler's estimate and from the chain's current base fee, with a buffer. The account needs ETH for that gas. There is no paymaster in this version. Estimation uses a simulation stand-in that is not the account and is not deployed. The fixed gas numbers used in tests are not a quote.

The figures below are Forge measurements on a local EVM, compiled with solc 0.8.28 and optimizer 200, against EntryPoint v0.9. They include the test harness around the call. They are not a mainnet fee.

Action	Gas
Create the account	223447
Send ETH and rotate the key	194768
Recover directly	28417

The same path sits on a few published standards. Naming them is the shape of the build, not a second specification.

Path	What it uses
Spending	ERC-4337 through EntryPoint v0.9, on one lane
Recovery	A direct transaction. It does not need the EntryPoint or the account's ETH
Logins	ERC-7739 against the current key. They die when that key rotates
Order grants	ERC-1271 for one exact digest, until expiry or until recovery
The account itself	A proxy. The code changes only by a recovery-signed upgrade

5. Recovery and the optional waits

Recovery is an ordinary transaction. The caller pays gas. It does not need the spending key, the EntryPoint, or an ETH balance on the account. Instant recovery is the default, because the usual case is a lost phone. A mandatory wait would make that case worse.

Which wait applies

Lost phone, words safe

Instant recovery. The words install a new spending key. This is the default.

Words stolen, phone safe

An optional delay lets the current spending key veto. Off unless you turn it on.

Words and phone both taken

The thief can veto you. That stalemate is why the delay is not mandatory.

Upgrade

Recovery only, at least seven days, and the new code is checked again when it runs. The spending key's remedy that week is to move the funds. There is no company key.

The recovery key signs a typed message bound to this account, this chain, and this recovery epoch. On success the account installs the new spending key and advances a security epoch, which invalidates order grants. If the request names the next recovery address derived from the phrase, that address becomes the authority and the epoch moves forward, so an old recovery signature cannot be reused. A hardware wallet with a single key can instead pass the zero address and keep the same authority; the mnemonic client is not allowed to do that.

Instant recovery is the default. An owner can set a waiting period. While it is non-zero, replacing the spending key waits, and loosening the spend policy waits. The current spending key can veto a pending recovery or a pending loosening. Lowering the delay is itself a loosening, so it cannot be used to skip the wait.

The tradeoff is the stalemate. With no delay, stolen words are immediate loss. With a delay, a person who still has the phone can veto a thief who has the words. If the thief has both the words and the phone, the thief can veto the real owner. That is why the delay is off unless the owner turns it on. The primary case is a lost phone, and a mandatory wait would make that case worse.

Upgrades follow the same hierarchy. Only a recovery signature can schedule one. The delay is at least seven days. The signed payload includes the new implementation and the hash of its code, and both are checked again when the upgrade runs. `cancelUpgrade` is also a recovery signature. The spending key cannot schedule an upgrade, cannot cancel one, and cannot execute one. There is no company allowlist and no pause switch. If a recovery key the owner no longer trusts schedules an upgrade, the spending key's remedy during that week is to move the funds. Assets that cannot be moved have no other remedy. Replacing the recovery verifier, which is the path for a future signature scheme, is an upgrade of this kind. It is not a setting the spending key can flip.

A stolen phone is cut off by entering the words once on a phone you still have. That sitting prepares every chain the phone can read, including every step of a chain that is several behind. The account still takes one step at a time, and those steps are paid in order. The previous phone loses a chain when that chain includes its own switch. Until then, it can still spend, and it can still get a transaction in first. What lands first stays moved. The words are cleared when you finish, go back, or leave that screen, and they are not saved. A chain that was never set up can be opened by whoever lands first. If it holds native funds, this phone submits that first use itself, paid from the balance. A chain that holds only tokens still needs someone to pay, which is what lets you

come back when the wallet's own ETH is already gone. That race is not gone. Approvals already sitting in token and NFT contracts stay there. On a chain that is already deployed, anyone can still pay the recovery fee.

6. What rotation does not erase

Retiring a spending key ends that key. It does not walk the rest of the chain and undo what other contracts already remember.

What survives the old key

Login signature
Dies on rotation. A retired key is not a general signer.

Token approval
Stays. It lives in the token contract. Rotation does not write there.

NFT operator grant
Stays. It lives in the NFT contract. Disconnecting a site does not revoke it.

Order grant
Stays, if you registered that exact digest on the account. Recovery ends it. A listing in this version is not active.

An ERC-20 allowance lives in the token contract. An NFT operator grant lives in the NFT contract. Retiring a spending key does not write to those contracts. An approved spender can keep moving assets without a new signature from the wallet. Rotation and allowance management are separate jobs.

The account can reject `approve(address,uint256)` of the maximum amount, and that switch is off by default. Leaving it off is a closed decision. The default policy is all zeros: no ETH cap, no large-transfer delay, no recovery delay, unlimited approvals allowed. Turning the rejection on matches that one selector. A token with a different spending function is outside that check. Rotation cannot erase an allowance that already lives in a token or NFT contract, and this version does not scan the chain for them.

Commerce is a separate rule. A swap or a bridge approves the exact amount of the token being sold, and never the maximum. An exact approval can still leave a remainder when the token does not debit the full amount.

The account accepts a standard ERC-721 or ERC-1155 safe transfer, so a token sent that way can land. Whether a marketplace will accept an ERC-1271 grant, and whether a login still works for that site after rotation, are not demonstrated. Listings stay inactive.

Signatures the hot key produces for logins die when the key rotates. That is the safe default: a retired key is not a general signer. A marketplace order is different only when the owner has registered that exact order digest onchain, naming the contract allowed to ask about it and an expiry. That grant survives rotation. It dies when recovery increments the security epoch, and the owner can revoke it. Registering the grant costs a transaction. A valid grant means the account

will recognize that digest. It does not mean the marketplace will fill the order. Listings are not active in this version, and they stay off until a test shows that a marketplace will accept this grant.

7. Optional commerce

Commerce sits after the free core. It is optional, and the fee appears only here.

Swap asks two providers for a same-chain route and keeps the better one. Bridge asks LI.FI for a cross-chain route. The 0x path is the AllowanceHolder flow: a plain token approval, then a call to the AllowanceHolder. Permit-style signatures are dropped. The spending key rotates before the calls run, so a permit signed by that key would be invalid by the time the swap checked it.

What the phone checks before it will sign

> **Where it ends**
> The route ends at the pinned AllowanceHolder or the pinned LI.FI Diamond, and every approval spends into that same contract.
>
> **The approval**
> Exact amount of the token being sold. Never unlimited. A required reset to zero is its own call.
>
> **Who receives**
> The receiver, the tokens, and the minimum are read from the route itself, not from a caption the service can rewrite.
>
> **The 0x Settler**
> Named inside the route and checked against the 0x registry. A paused registry, or an unnamed Settler, stops the signature.
>
> **Your limits**
> The spend policy is read first. A quote that would trip an ETH cap or a delay is refused before signing.
>
> **The simulation**
> The batch is run in order. Below the quoted minimum, the phone will not sign. If the network cannot simulate, the sheet says only a gas check ran.

Fee rule	Value
Default	0.25 percent, on an optional swap or bridge only
Cap	0.30 percent. Anything above that is rejected
No fee recipient	Zero. Not an error
What decides the winner	The minimum you receive, then the network fee. The KNOAD fee is shown and does not rank the quotes
How long a quote lives	30 seconds, and not past its own deadline. A refreshed quote has to match the calls on the sheet
Submission	Off until the account review. The sheet shows the quote and sends nothing

The quote service holds the provider API keys. The phone treats its response as untrusted and runs the same checks again:

- The route ends at the pinned 0x AllowanceHolder or the pinned LI.FI Diamond, and every approval spender is that same contract.
- Approvals are for the token being sold, for the exact amount, and never unlimited. A token that must be reset to zero first gets that reset as its own call. An exact approval can still leave a remainder when the token does not debit the full amount. Ordinary `approve` outside commerce stays unlimited unless the owner opts in.
- The receiver, the tokens, and the minimum are read from the provider's own calldata, not from a field the service can rewrite.
- For 0x, the Settler contract named inside that calldata is checked against the 0x registry onchain. A paused registry, or a Settler the registry does not name, stops the signature.
- The account's spend policy is read first. A quote that would exceed the ETH limit or the delay threshold is refused.
- The whole batch is simulated in order, so the approval is visible to the swap. If the simulated amount received is below the quoted minimum, the phone will not sign. If the network cannot simulate, the sheet says that only a gas check ran.

Quotes are ordered by the minimum the user receives. When two are within a tenth of a percent, the lower network fee wins. The KNOAD fee is recorded and displayed, and it never decides the winner. The default is 25 basis points. Anything above 30 is rejected in the phone and in the quote service. An unset fee recipient produces a zero KNOAD line, not an error. A quote older than 30 seconds, or past its deadline, is fetched again before signing, and the calls that get signed have to match the calls on the sheet.

`EXPO_PUBLIC_COMMERCE_SUBMIT` is off in the development, preview, and production profiles. Off means the phone shows the quote and says that submission opens after the account review. Nothing is signed and nothing is sent. When a bridge is eventually submitted, the phone follows it from the bundler receipt to the destination chain and shows it as in transit until it lands or fails.

LI.FI collects its integrator fee into its own fee collector, which KNOAD claims later. 0x pays the named recipient inside the quote. Buy / Sell, if it is added later, will be a hosted partner flow. KNOAD will not handle card data or identity checks. That product has no fee in this version because it is not active.

8. If KNOAD is gone

The account and the published factory addresses do not depend on a KNOAD server to exist, to send, or to be recovered. The quote service is the only piece whose absence removes a feature, and it removes only swaps and bridges. No chain has the factory yet, so there is no deployed bytecode to compare with the published addresses. The repository includes a command-line

recovery command that builds the call and can broadcast it when given a payer key. It is not a hosted recovery service. The phone builds the same kind of call.

What still works

Finding the address
The words, the account index, and the published factory. No KNOAD index. A lost phone can also find an already-deployed account from the factory's public event on any RPC. The factory address is published, and no chain has that code yet.

Sending
Another bundler can submit a signed operation.

Recovering
Any account that pays the network fee can submit it. It does not call KNOAD.

Prices
Swap and bridge quotes stop. Send, recovery, and pairing do not.

The company
Not a party to validation or recovery. No admin key can freeze the account, replace the signer, or upgrade it.

The account address can be recomputed from the 24 words, the account index, and the published factory and implementation addresses. No KNOAD index is required. A client that has lost the phone can find an already-deployed account from the factory's public event on any RPC, once a chain has the factory. This version's phone derives with an empty BIP-39 passphrase. A passphrase is not a switch that can be turned on after the address exists.

Sending and recovery do not call a KNOAD server. Another bundler can submit a signed operation. Recovery can be submitted by any account that pays the gas. The quote service is the one piece whose absence removes swaps and bridges, and only those.

Illuminary Labs is not a party to validation and not a party to recovery. There is no admin key that can freeze the account, replace the signer, or upgrade the implementation.

9. What this is not

An independent review of the account, the factory, the recovery verifier, and the phone that holds the keys has not been done. The project's own tests are evidence the authors can show. They are not that review.

The keys are secp256k1 and P-256. One successfully included spending operation retires the previously authorized key. That narrows a window. It does not make the account post-quantum. A recovery signature publishes the recovery public key, as any ordinary signature does.

A person holding the 24 words has the recovery authority. This version derives them with an empty BIP-39 passphrase, chosen by the absence of a prompt. A passphrase would have been a different address, and it would have had to be chosen before that address existed. The chain

cannot tell the holder of the words from the owner. A compromised phone can sign a theft, and the contract will obey that signature while the key is current on that chain.

Existing token approvals and NFT operator grants remain until they are revoked in those contracts. Disconnecting a site does not do that revocation.

Swap and bridge submission is off. Buy / Sell and NFTs do not transact. There is no paymaster, no KNOAD token required to recover, and no subscription required to create, send, or restore.

The register below is the outcome of each failure the design is written against. The source file is the threat model docs/threat-model.md. It is not an independent verification.

Failure	What the design does	What remains
Spending key stolen	An included operation retires it on that chain. Other chains keep the key they already have until each includes its own change. Entering the words once, on a phone you still have, prepares every chain that phone can read.	A thief can race, and can spend while the key is current, including on a chain that has not included its own change. What gets included first stays moved. Malware that watches the phone sees the next key too.
24 words stolen	The words are meant to stay offline. An optional delay lets the current key veto. This version uses an empty BIP-39 passphrase.	The words are full ownership of this address. No delay means instant loss. Delay plus a stolen phone can deadlock. A passphrase added later would be a different address.
Spending key tries to change the way back	It cannot. Recovery replaces the signer.	Transfers that already landed stay landed.
Next key lost before use	The phone must save it before signing. Recovery still works if that copy is gone.	A client that signs first can lock the spending path.
Stale sequence or a dropped operation	The phone reads the account, not a bundler’s “accepted.”	A censored chain still does not include transactions.
Standing token approval	Rotation does not pretend to revoke it. Unlimited approval can be rejected if you turn that on.	Other spending functions, and unknown spenders, remain.
A marketplace order dies on rotation	An exact onchain grant survives rotation.	Registering it costs a transaction. A revoked approval can still make the order unfillable.
A login is replayed	Live signatures are checked against the current key. Grants are bound to this account, this caller, and this recovery era.	Pointing the check at the wrong contract approves the wrong contract.
Upgrade takeover	Recovery signature, at least seven days, the new code checked again, no admin key. `cancelUpgrade` is recovery-signed. The spending key cannot cancel.	A stolen recovery key can schedule an upgrade. The spending key’s remedy is to move the funds during the wait. Assets that cannot be moved have no other remedy.
Factory front-run	The address commits to the original recovery authority, the index, and the empty creation	Each chain’s first spending key is live there until that chain includes an

Failure	What the design does	What remains
	policy. A different spending key is installed only when that recovery key signed it for that chain.	operation. Whoever lands first on a chain that was never set up chooses the key.
A hostile site or a swapped clipboard	The signing sheet is on the phone, outside the page.	A compromised phone can still sign.
A quantum break of today's signatures	One successfully included spending operation retires the previously authorized key. The recovery check can be replaced later by an upgrade.	This version is not post-quantum. A recovery signature publishes a recovery public key, as any ordinary signature does.
A stolen unlocked phone, on a chain not yet deployed	The phone stores a recovery-signed activation and the derived first key in the password-encrypted vault, then discards the words. The activation dies when that chain is deployed. If the chain holds native funds, recovery submits the first use itself and installs a new key, paid from that balance. An optional secure-all step lists every chain and prepares an outside payment for an empty chain or a chain that holds only tokens.	Restoring the words rebuilds the same activation, so it does not cut off the old phone. Until recovery lands first, that phone can open the chain and take what was received there. Tokens with no native balance still need someone to pay the deployment. Until that outside payment lands, the old activation still works. The race is not gone.
A bundler refuses the operation	Another bundler can submit it. Recovery can be submitted directly by anyone who pays the fee.	Chain-level censorship still wins.
A fatal bug in the account	The custom surface is small. The owner can schedule an upgrade.	There is no pause switch. A bug in the recovery path itself may be unrecoverable.
Quote service compromised	The phone checks the route again: pinned contracts, no permits, and the 0x Settler against the registry.	A bad service can offer a worse price, go silent, or mis-state a fee the route does not pin down. A bridge's destination-side minimum is not fully readable.
Estimation stand-in	Used only to price a transaction. It is not the account, and it is not deployed.	Installing any new account code still takes the recovery key.

10. Where the work stands

The account, factory, recovery verifier, phone, keyless extension, and quote service described above are implemented. Spec 1.4's open choices — spend limits, recovery delay, and upgrades — are implemented as specified there: off by default, owner-controlled, and unreachable by the spending key. Features that are not built, including permission discovery, individual NFT listing approvals, marketplace compatibility, persistent website sessions, and external wallet portability, are in the roadmap roadmap.md.

Evidence	State
Account rotation, recovery, policy, grants, and upgrades	Covered by the project's own tests

Evidence	State
The phone saves the next key before signing, and a paired session cannot see the phrase	Covered by the project's own tests
Commerce rejections, ranking, the fee cap, and the policy precheck	Covered by the project's own tests
Quote service fee handling, and refusal of permit routes	Covered by the project's own tests
Gas estimation stand-in	Covered as a simulation that is not the account and is not deployed
Foundry suite without a fork URL	Passing
Fork run of a recorded live swap	Not run
Live provider quote, end to end	Not run
Commerce submission	Off until review
One entry prepares every chain the phone could read	Looking needs no transaction. A replacement sitting signs every recovery step in order, including a chain several steps behind. The account still accepts one step at a time, and that limit is covered by the project's own tests. A live inclusion on all four chains has not been run
First use on an old device, a restored device, and a newly added network; a stolen phone that loses the race; a repeated next key in one bundle; an empty account and a token-only account left for an outside payer; rotation, speed-up, cancel, a send that lands before its replacement, and a reverting send	Shown locally, on anvil, with EntryPoint v0.9 and a bundler enforcing its validation rules. The record is `test-vectors/live-report.json`. Nothing was broadcast. The phone screens were not clicked through. P-256 was not part of that run, because that anvil does not serve the precompile. The Forge P-256 test still covers the key check
Factory bytecode on a chain	Not broadcast. The published addresses are what the clients compute against
Recovery included on Ethereum, Base, Sepolia, and Base Sepolia	Not done
Live swap and bridge	Not done. Submission stays off
Independent review of the account, the factory, the recovery check, and the phone that holds the keys	Not done. This is the release gate

A LI.FI bridge's destination-side minimum is not fully readable from the shared bridge data, so that residual is disclosed rather than claimed away. A live route that sells native ETH has not been checked against the provider; a route the phone cannot decode is refused, not signed.

The release gate is unchanged. Real assets wait on that review. Until then, these contracts are not cleared to hold assets of value.

Appendices

Each appendix opens with the one-sentence version. The body stands without them.

Appendix A. Glossary

The words used in the body, pinned down.

Term	Meaning
Address	The smart account. It stays when the spending key changes.
Hot signer	The one spending key that is current.
Generation	The count of included hot operations. It is also the account's transaction sequence.
Recovery authority	The address derived from the 24 words. It can install a new hot signer.
Write-ahead	Saving the next spending key before the current key signs.
Inclusion	The chain accepted the operation. A bundler's "received" is not inclusion.
Grant	An onchain record that a named contract may rely on one exact order digest until expiry or until recovery.
Paymaster	A contract that pays gas for a user. This version has none.
Quote	A priced route the phone has checked. Showing a quote is not a submission.

Appendix B. Chains and published addresses

These are the addresses the clients compute against. No chain has the factory yet. They are not an invitation to send assets before the review in section 10.

The account address is the same on all four chains because the factory, the implementation, and the proxy bytecode are the same and the salt commits to the original recovery authority, the account index, and the empty creation policy. The first spending key is not part of it.

Chains	Ethereum (1), Base (8453), Sepolia (11155111), Base Sepolia (84532)
EntryPoint v0.9	`0x433709009B8330FDa32311DF1C2AFA402eD8D009`
Recovery verifier	`0x39305881423DC27C6238bE3A343787772534e764`
Factory	`0x640b63604581cC0b1326025a57C188BCc6160Dc9`
Implementation	`0x70044945917051ce6DcCD244C4d4288FE14A9f28`

P-256 verification uses the chain precompile. All four chains in the registry have it. A chain without it is not advertised as supported. The source of these values is `test-vectors/deployments.json`, produced by the deployment script.

Appendix C. Derivation

The phrase derives the recovery key and the first spending key. Later spending keys are random and stay on the device.

Recovery material is a 24-word BIP-39 mnemonic. This version derives with an empty BIP-39 passphrase. The phone never asks for one, so the empty passphrase is the choice, made by the absence of a prompt. A passphrase would have to be chosen before the address exists. Choosing one later would derive a different address. It is not a setting. The app password only encrypts the vault.

The paths are hardened and sit outside `m/44'/60'`, so importing the phrase into a typical Ethereum wallet does not expose the recovery key.

Use	Path
Recovery epoch `e` of account `a`	`m/7331'/60'/a'/0'/e'`
One-time initial hot key of account `a` on chain `chainId`	`m/7331'/60'/a'/1'/<chainId>'`

The recovery epoch on the account is that index. It starts at 0. Clients read it from the account. They do not scan.

The first spending key is secp256k1 and derived, one path per chain, so a phone can spend on a chain that is still on that key before a device key exists. The path includes the chain id. The address does not. The first included operation on a chain replaces that chain's key only. Later spending keys are fresh, one per chain, device-bound when the platform allows (Secure Enclave or StrongBox, non-exportable, not synced), and are not derived from the phrase. The account index in the phone is 0.

Appendix D. Contract behavior

A map of the account's behavior. Spec 1.4 is the normative text. This appendix is not a second specification.

`executeAndRotate` is the hot path. It runs only from the EntryPoint, only after validation has rotated the key and bound that operation to its own next signer. Generic execute reverts. A zero-length execution payload is a key refresh and still rotates. An encoded batch of length zero reverts. `vetoRecovery` is the other hot-key operation: it cancels a pending delayed recovery or a pending loosening of policy, and it rotates like any other included hot operation. The account inherits the standard ERC-721 and ERC-1155 receiver hooks, so a safe transfer of either kind can land.

`recover` checks the recovery signature, increments the authority nonce, and either installs the new hot signer immediately or stores the request until the delay has passed. Anyone can submit it, and anyone can apply a request whose delay has elapsed. A second use of the same signature fails.

`setPolicy` requires a recovery signature. Tightening applies immediately. Loosening waits out the current recovery delay and can be vetoed. A period limit without a window, or a delay threshold without a delay, is rejected. The ETH cap counts native value across the batch.

`approveOrderDigest` and `revokeOrderDigest` are the only calls the account may make to itself. `isValidSignature` accepts a digest when a grant names the caller, the grant is unexpired, and the grant's security epoch is current. Otherwise it checks the current hot key under ERC-7739. Personal-sign and ordinary typed-data logins therefore die on rotation.

`scheduleUpgrade` requires a recovery signature, a delay of at least seven days, and the codehash of the new implementation. `cancelUpgrade` requires a recovery signature against the pending implementation. The spending key cannot call it. `executeUpgrade` checks the codehash again. `upgradeToAndCall` from anywhere else reverts.

The factory's salt is the hash of the original recovery address, the index, and the hash of the creation policy. The proxy is created with `initialize` of those same values. The first spending key is installed in that same transaction only when the epoch-0 recovery key signed it for that chain. That signature has no expiry. A different signer, or a signature from another chain, does not deploy. If the account already exists, the call returns it and does not change the key. Calling `initialize` on the implementation reverts.

Appendix E. Commerce checks

The phone will not sign a route that fails these checks. The quote service failing them means the route is dropped before it is shown.

Pinned targets, checked against the providers' docs and the explorers on 15 October 2026:

Contract	Address
0x AllowanceHolder	`0x0000000000001fF3684f28c67538d4D072C22734`
LI.FI Diamond	`0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE`

The 0x Settler is not pinned. 0x rotates it. The phone reads the Settler out of the AllowanceHolder calldata and checks `ownerOf(2)` on the 0x registry at `0x0000000000004533Fe15556B1E086BB1A72cEae`, falling back to `prev(2)` during the window when the API still targets the previous deployment. Permit2 (`0x000000000022D473030F116dDEE9F6B43aC78BA3`) is rejected wherever it appears.

A route of zero calls, or of more than eight, is rejected. So is a self-call, a call to the zero address, a call that is not an approval placed before the provider call, an approval to a different provider than the one the route calls, and a receiver that is not the account. Native sales carry the sell amount as value and carry no approval. Token sales attach no native value and carry one exact approval.

The spend-policy precheck reads the account's policy before the phone will sign. An undeployed account has the zero policy. A deployed account whose policy cannot be read is refused, rather

than treated as unlimited.

Gas estimation uses a simulation stand-in. It is not the account, and it is not deployed. Installing any new account code still takes the recovery key.

Two limits remain visible on purpose. A LI.FI bridge's destination-side minimum, and any receiver field that lives only inside one bridge's private data, are not fully readable from the shared bridge struct. The loss in that case is bounded by the amount being sold, and the receiver in the shared struct still has to be the account. A live 0x route that sells native ETH has not been confirmed against the API. If the calldata does not match the shape the phone knows, the quote is refused.

Appendix F. Residual risk

The register is docs/threat-model.md. It describes what the implementation is written to enforce. It does not claim an independent verification.

The register's T17 is a stolen unlocked phone on a chain that has not been deployed. The stored activation has no deadline. Recovery that finds native funds submits first use itself. An optional secure-all step lists every chain and prepares an outside payment for an empty chain or a chain that holds only tokens. Restoring the words does not cut the old phone off. Tokens with no native balance still need someone to pay. Until that payment lands, the old activation still works. The race is not gone.

Two rows were added for this version:

- A compromised quote service can offer a worse price, can be unavailable, or can mis-state a fee the calldata does not pin down. It cannot add an unpinned call, redirect the receiver, or attach a permit, because the phone checks those itself.
- Gas estimation uses a simulation stand-in. It is not the account, and it is not deployed. Installing any new account code still takes the recovery key.

The older note that a desktop extension was out of scope is updated in that register. The extension exists. It holds no keys. An extension that stored a seed would be a different product, and it is not this one.

Appendix G. What changed from draft 1.2

Draft 1.2, the PDF beside this file, described a proposed wallet and listed open engineering decisions. This paper describes the implementation.

Draft 1.2	This paper
Recovery called unimplemented	Recovery is implemented on the phone and in the account
Factory, discovery, upgrades, and gas left open	Closed in spec 1.1 and in the code, and described in sections 4, 5, and 8
Allowances limited by default	Default policy allows them. Unlimited-approval rejection is opt-in

Draft 1.2	This paper
Rotation described as following a successful outer operation	Rotation happens when validation succeeds, including when the calls revert
Companion extension treated as part of a future wallet	The extension is a keyless relay
Single-chain posture	One address on Ethereum, Base, Sepolia, and Base Sepolia
No commerce	Quotes, checks, and simulation exist. Broadcast is off until review

The draft's warning still holds: do not use these contracts with assets of value until the review in section 10 is done.